

AWS Developer Exam: Domain 3 – Automate Deployment Testing

Overview

This task statement focuses on automating the deployment testing process using AWS services. Key themes include:

Infrastructure as code and configuration as code fundamentals

Automated deployment strategies with CodeDeploy and SAM

Creating and managing test events for Lambda

API Gateway staging and environment differentiation

CI/CD pipeline automation with CodePipeline

Cross-account deployments using IAM roles

Infrastructure as Code vs. Configuration as Code

Two foundational concepts to understand before automating deployments:

Concept	Tool(s)	Purpose
Infrastructure as Code (IaC)	CloudFormation, CDK, SAM	Define and provision AWS infrastructure through templates
Configuration as Code	Chef, Puppet via OpsWorks	Manage software configuration and application setup on servers

CloudFormation Depth (Must Know)

Stacks — A collection of AWS resources managed as a single unit

Change sets — Preview how proposed changes to a stack will affect running resources before applying them

Permissions — IAM roles required for CloudFormation to create and manage resources

Template structure — AWSTemplateFormatVersion, Description, Parameters, Mappings, Conditions, Resources, Outputs, Transform

OpsWorks and Elastic Beanstalk

OpsWorks — Use when configuration tasks are better handled with **Chef or Puppet**; supports automated configuration management for EC2 instances

Elastic Beanstalk — Simplified deployment and management of applications; know its deployment best practices for the exam

CodeDeploy Deployment Strategies for Lambda

AWS CodeDeploy automates application deployments to EC2 instances, on-premises servers, Lambda functions, and ECS. For **Lambda deployments**, the configuration specifies how traffic is shifted to new versions.

Lambda Deployment Strategy Types

Strategy	How It Works	Use When
All-at-Once	Shifts 100% of traffic to the new version immediately	Fastest; highest risk
Canary	Shifts traffic in two increments — a small percentage first, then the rest after validation	Gradual validation with two steps
Linear	Shifts traffic in equal increments with an equal number of minutes between each increment	Steady, time-based rollout (e.g., every 15 minutes)
Immutable	Not applicable for Lambda deployments	N/A

Exam Scenario: Lambda functions deployed via SAM + CodeDeploy, shifting traffic every 15 minutes until all traffic is on the new version → **Linear deployment**. Keywords: equal increments, equal intervals.

Creating Test Events for Lambda

To make local development and testing of Lambda functions easier, you can generate and customize **event payloads** for a number of AWS services, including API Gateway, CloudFormation, S3, and others.

How Lambda Receives Payloads

API Gateway maps HTTP requests to a **JSON object**, which becomes the payload sent to the Lambda function.

You can create a REST API with a resource and method in API Gateway, back the method with a Lambda function, and invoke the function by calling the API's HTTP endpoint.

Advanced API Gateway + Lambda Capabilities

Full request passthrough — Pass the entire request to Lambda without modification

Catch-all methods — Use the `ANY` method to catch all HTTP methods on a resource

Catch-all resources — Use a **proxy resource** to catch all paths beneath a resource

Creating Test Events in Lambda

You can create **test events directly in the Lambda console** to simulate how different event sources would invoke your function — without needing a real upstream trigger.

API Gateway Staging and Environment Differentiation

Core API Gateway Concepts

Concept	Description
Stage	A named reference to a deployment; a snapshot of the API at a point in time
Resource	Defines a path in the API (e.g., <code>/users</code>)
Method	HTTP operation on a resource (e.g., GET, POST); backed by an integration
Integration	Routes method requests to a Lambda function or other backend
Deployment	A published version of the API associated with a stage

Differentiating Staging and Production Environments

Options for creating separate environments in API Gateway:

Rate limiting per stage — Restrict access based on throttling rules in staging

Method-level access control — Restrict which methods are available in each stage

Single endpoint with stages — Use multiple stages (dev, test, prod) under one API

Separate API services per environment — Maintain completely isolated APIs for each environment

Proxy Integration vs. Custom Integration

Integration Type	Behavior
HTTP_PROXY	API Gateway passes the entire request to the backend HTTP endpoint without intervention; response is passed back as-is
HTTP (Custom)	API Gateway can transform the request and response using mapping templates
AWS_PROXY (Lambda Proxy)	API Gateway passes the full request to Lambda as a structured JSON event; Lambda returns the full response

Exam Scenario: If you need API Gateway to pass client-submitted requests to the backend **without intervention from the gateway** → use **HTTP_PROXY** or **AWS_PROXY** (Lambda Proxy) integration.

Automating the CI/CD Pipeline with CodePipeline

Why Automation Matters

Deploying code in a repeatable fashion while reducing manual error requires **automating the entire release process**. CodePipeline orchestrates each step in the release process.

CI/CD Pipeline Stages

Stage	Description
Source	Code is committed to a repository (e.g., CodeCommit, GitHub)
Build	Code is compiled, tested, and packaged (e.g., CodeBuild)
Test	Automated tests are run (unit tests, integration tests)
Staging	Application is deployed to a pre-production environment
Production	Validated code is deployed to the production environment

Automating Software Testing

Create **test events in Lambda** to simulate invocations from different event sources.

For serverless applications, **automated tests are essential** as they scale — they provide real-time or fast feedback on the state of your application.

Use **unit testing** and **mock testing** as part of your automated pipeline.

Container Image Tagging and Environment Management

Amazon ECR — Image Tagging Strategy

Tagging container images in ECR helps manage different environments:

Tag **development images** and push to a **dev repo**

Tag **production images** and push to a **prod repo**

Use tags to **group repos by development team**

Apply **IAM policies** to secure push and pull access per environment

AWS Amplify

Amplify is a set of tools and features for frontend web and mobile developers to build full-stack applications on AWS.

Amplify Component	Purpose
Amplify Hosting	Git-based workflow with continuous deployment ; updates deploy on every code commit
Amplify Studio	Visual development environment for full-stack web and mobile apps

Key capabilities of Amplify Hosting:

Deploying **serverless backends** with GraphQL or REST APIs

Built-in **authentication, analytics, and storage**

Automatic deployments triggered by **code commits**

AWS Copilot

Copilot deploys and operates **containerized applications on ECS** directly from source code.
Deploy through a **CodePipeline pipeline** across multiple environments, accounts, and Regions
Manage the entire deployment lifecycle from the **CLI**

Cross-Account Deployments with CodePipeline

Use Case

Deploy Lambda APIs across different AWS accounts and environments (e.g., pre-production and production) using **cross-account IAM role assumption** (STS AssumeRole).

Example Pipeline Architecture

Stage	Action
Commit	Code is pushed to source repository
Build	CodeBuild creates a CloudFormation template and saves the template stack to S3
Deploy (Pre-Prod)	CloudFormation deploys the application code from S3 using API Gateway endpoints; IAM pre-prod role assumed
Deploy (Production)	Application code changes are deployed by assuming the STS IAM production role

Required IAM Roles and Policies

Role / Policy	Purpose
Cross-account role	Allows CodePipeline to assume a role in the target account
S3 read/write policy	Allows the pipeline to read/write deployment artifacts in S3
KMS key access policy	Allows access to encryption keys in the production account
Development role for CloudFormation	Allows CloudFormation to create and manage resources in the dev/pre-prod account
Lambda function service invocation policy	Grants permission to invoke Lambda functions
API Gateway invocation policy	Grants permission to invoke API Gateway endpoints

Key Exam Tips

Linear deployment shifts traffic in equal increments at equal time intervals — the right choice for "every X minutes" scenarios with CodeDeploy + Lambda.

Canary = two increments. **Linear** = many equal increments. **All-at-once** = immediate. **Immutable** does not apply to Lambda.

API Gateway maps HTTP requests to a JSON object — that JSON becomes the Lambda event payload. The `ANY` method catches all HTTP methods; a **proxy resource** catches all paths beneath a resource. Use **HTTP_PROXY** or **AWS_PROXY** when API Gateway should pass requests to the backend without modification.

Stages in API Gateway are snapshots of a deployment — use them to maintain separate dev, test, and prod environments under one API.

Test events in Lambda simulate upstream triggers — use them to automate testing without real event sources.

AWS Amplify Hosting = git-based, continuous deployment; auto-deploys on every commit.

AWS Copilot = deploy containerized apps to ECS from source code via CodePipeline, across accounts and Regions.

Cross-account deployments require a cross-account IAM role, S3 read/write policy, KMS key access, and service-specific invocation policies in the target account. EOF